Rotaract Club of Xiamen University Malaysia | Rotary Club of Kuala Lumpur Dynamics | Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: [Insert date]

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	Catalyst
Project / Solution Name	Orbyte SlipMatch - Instant Chat-to-Bill Generator & Automated DuitNow Receipt Validator
Name (Leader)	Ang Yi Xian
Email & Phone (Leader)	xian12053@gmail.com & +601110661817
University Affiliation(s)	Universiti Malaysia Perlis
Live Prototype URL	https://sdc-2026-catalyst.replit.app
Code Repository URL (public / access granted to judges)	https://github.com/xian3512/SDC2026---Catalyst/

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

Orbyte SlipMatch is a web-based order management and payment verification system built using HTML, CSS, JavaScript and Python FastAPI, with Replit as its development and hosting platform. The workflow begins when an owner pastes a customer's WhatsApp order in Bahasa Melayu, English or Manglish, or when staff upload a payment receipt. Order information is extracted using Gemini AI with a local parsing fallback, while product prices and bill totals are calculated using predefined rules. Confirmed orders are assigned unique order IDs and references, with product information, orders, payment records and audit logs stored in SQLite. Uploaded payment receipts are processed through an OCR component configured to use RapidOCR on RunPod Serverless, with fallback options available. Rule-based matching evaluates payment amounts, transaction references, duplicate payments, short payments and possible multiple-order combinations. The system presents the findings through a Green, Yellow or Red classification dashboard, allowing the owner to approve, hold or reject transactions. All review decisions are recorded in the audit log, while WhatsApp reminders are prepared for manual sending. Three-way reconciliation against imported synthetic bank records is part of the proposed workflow, although mandatory bank-credit verification has not yet been demonstrated in the reviewed implementation.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
Python 3.14.5 (local); HTML5, CSS3, JavaScript t	Backend development, web dashboard, and owner/staff interfaces	PSF; W3C/ECMA standards
FastAPI; Uvicorn 0.54.0	HTTP endpoints, hosted dashboard, multipart requests. Installed versions need runtime check.	MIT; BSD-3-Clause

Name & Version	Purpose in Project	License
Pydantic; python-multipart	API models, validation, image uploads and decisions.	MIT; Apache-2.0
Pillow 12.3.0; Requests 2.34.2	Receipt image validation; RunPod/Gemini REST calls.	MIT-CMU; Apache-2.0
RapidOCR; ONNX Runtime	Pre-trained OCR inference; RunPod worker when available.	Apache-2.0; MIT
SQLite, sqlite3 and Python stdlib	Catalogue, invoices, slips, bank-statement table, audit log; exact-cent calculation.	Public domain; PSF

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
Google Gemini API (Gemini 3.8 Flash)- Google	AI-assisted extraction of customer names, products, quantities and order details from Bahasa Melayu, English and Manglish WhatsApp messages for automatic bill generation.	Free
RunPod Serverless API- Runpod	Cloud-hosted RapidOCR service for extracting payment amounts, dates, and transaction references from uploaded receipts. Successfully tested using the Serverless API.	Sponsor-provided
WhatsApp click-to-chat (wa.me)- Meta	Prefilled short-payment reminder link opened by owner; no WhatsApp Business API.	Free external link
Bank data feed / payment gateway	None. The prototype uses synthetic/local bank-statement viewer, not a live bank API.	None

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
Replit (development host)	Development and integration of the frontend, backend APIs, dashboard, and database components.	Plan and deployment region to confirm
RunPod Serverless	Cloud computing service hosting the RapidOCR worker for extracting payment amounts, dates, and transaction references from uploaded receipts. Successfully tested through API calls.	Serverless CPU (2 vCPU, 8 GB)

Service & Provider	Purpose in Project	Plan / Region
SQLite local file	Main app database with bakery menu, orders, slips, matches, audit and bank table.	App filesystem; not SaaS DB
Docker / registry	Cloud container registry used to store and distribute the RapidOCR Docker image (angyx/orbyte-ocr:v1) for Runpod Serverless deployment.	Public image repository / Global registry

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
Gemini 3.8 Flash	Google DeepMind / Google Gemini API	Pre-trained	Google Gemini API Terms of Service
RapidOCR (version TBC)	RapidAI / PaddleOCR	Pre-trained OCR for receipt text; no custom fine-tuning.	Apache-2.0 (source + upstream weights)

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
Demo receipt fixtures	Team-created local fixtures	Synthetic	Team-created	No, if entirely synthetic
Demo product catalogue	Local SQLite database	Synthetic / team-created	Team-created	No, if entirely synthetic
Demo order records	Local SQLite database	Synthetic / team-created	Team-created	No, if entirely synthetic
Demo bank-statement records	Local or synthetic bank data	Synthetic	Team-created	No, if entirely synthetic

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
ChatGPT (GPT-6)	Main project development, backend programming, OCR integration, and debugging	Major
Claude Opus 5.5	Generating synthetic payment receipts for OCR testing and demonstration	Minor
Replit AI	Frontend, backend, and database integration	Moderate
Gemini 3.1 Pro	Main project development, code generation, debugging, technical problem-solving, and project documentation	Major
Google Flow	Creating AI-generated animations for the Orbyte promotional video	Moderate

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

The team reviewed and refined all AI-generated code, documentation, and visual content before incorporating them into Orbyte SlipMatch. Backend functions were tested using synthetic WhatsApp orders, payment receipts, and transaction records to evaluate amount matching, duplicate reference detection, and error handling. AI-assisted code was reviewed, debugged, and validated through local testing and API responses. Technical documentation was checked for accuracy, while AI-generated promotional materials were reviewed for relevance and appropriateness. All synthetic data were used exclusively for testing and demonstration

purposes. The team maintained responsibility for the final outputs, with human approval required before confirming payment transactions.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☑ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Runpod	RunPod Serverless for OCR inference; 5 worker	RapidOCR Serverless deployment and payment receipt OCR testing
Replit	FastAPI development/hosting; user Replit Core voucher.	Frontend, backend and database integration

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

Replit and RunPod Serverless played important roles in developing the Orbyte SlipMatch prototype. Replit provided an integrated environment for developing, testing and hosting the FastAPI application, while RunPod Serverless was selected to support cloud-based OCR processing of payment receipts. These resources simplified development, reduced infrastructure management and supported rapid prototyping within the hackathon timeframe. Although they contributed significantly to the development process, both platforms could be replaced by alternative hosting or computing services without fundamentally changing the system architecture or core functionality.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

The prototype stores order information, payment-slip metadata and matching results in a SQLite database. Uploaded receipt images are stored in the project's uploads directory. These records may contain customer names, payment amounts, transaction dates and bank reference details. Synthetic data should be used for demonstrations wherever possible. Access to project files and uploaded images should be restricted, and API credentials should be stored in environment variables rather than committed to source control. The current retention and deletion schedule has not been formally established and should be defined before real deployment. If receipt images or extracted text are sent to third-party AI or OCR services, the team should document the information transmitted and review the providers' applicable data-handling terms. Real customer information should not be published in the repository or shared unnecessarily.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

Receipt OCR may fail on blurry, cropped or low-resolution images. AI-based extraction may produce incomplete or incorrect fields, while external API availability, rate limits and model versions may change. Demo OCR configurations may recognize only specific sample receipts. Matching an amount or reading a transaction reference does not establish that money reached the merchant's bank account. The system therefore uses rule-based checks for amounts, references, duplicate submissions and short payments, and presents results for owner review. The owner should confirm actual bank credits before marking an order as paid. API integrations and model versions must be tested in the deployed environment, and detection accuracy should be measured on representative synthetic test cases before performance claims are made.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- ☑ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☑ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☑ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☑ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☑ Any use of official SDC sponsor resources has been declared in Section 5.
- ☑ The prototype described here is accessible to judges through the links provided in Section 1.
- ☑ All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.



[Ang Yi Xian]

[25 Oct 2026]